Real-Time Case Study: E-Commerce Order Analytics Using Java Streams

Scenario

You are part of an E-Commerce backend analytics team. Your system loads order data in-memory and management wants analytics reports using **Java Streams API** only — no loops allowed.

---

1. Domain Model

Order Class

```
package com.ecommerce.analytics;

public class Order {
    private int orderId;
    private String customerName;
    private String category;
    private double amount;
    private String city;
    private String status; // e.g., PAID, PENDING, CANCELLED

    public Order(int orderId, String customerName, String category, double
amount, String city, String status) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.category = category;
        this.amount = amount;
        this.city = city;
        this.status = status;
    }

    public int getOrderId() { return orderId; }
    public String getCustomerName() { return customerName; }
    public String getCategory() { return category; }
    public double getAmount() { return amount; }
    public String getCity() { return city; }
    public String getStatus() { return status; }

    @Override
    public String toString() {
        return "Order{" +
                "orderId=" + orderId +
                ", customerName='" + customerName + '\'' +
```

```
                ", category='" + category + '\'' +
                ", amount=" + amount +
                ", city='" + city + '\'' +
                ", status='" + status + '\'' +
                '}';
    }
}
```

Sample Data

```
List<Order> orders = Arrays.asList(
        new Order(101, "Ravi", "Electronics", 55000, "Delhi", "PAID"),
        new Order(102, "Sita", "Clothing", 12000, "Mumbai", "PENDING"),
        new Order(103, "Amit", "Furniture", 75000, "Delhi", "PAID"),
        new Order(104, "Priya", "Electronics", 30000, "Chennai", "PAID"),
        new Order(105, "Raj", "Clothing", 8000, "Delhi", "CANCELLED"),
        new Order(106, "Anita", "Furniture", 25000, "Mumbai", "PAID"),
        new Order(107, "Vikram", "Electronics", 95000, "Bangalore", "PAID"),
        new Order(108, "Meena", "Clothing", 60000, "Delhi", "PAID")
);
```

---

2. Stream Concepts with Simple Example and Case Study Mapping

1. Filter

Simple Example:

```
List<Integer> nums = List.of(10, 15, 20, 25, 30);
nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Case Study Example:

```
List<Order> paidOrders = orders.stream()
    .filter(o -> o.getStatus().equalsIgnoreCase("PAID"))
    .toList();
```

---

2. Map

Simple Example:

```
List<String> names = List.of("ravi", "sita", "amit");
names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Case Study Example:

```
List<String> paidCustomerNames = orders.stream()
    .filter(o -> o.getStatus().equalsIgnoreCase("PAID"))
    .map(Order::getCustomerName)
    .toList();
```

---

3. Reduce

Simple Example:

```
List<Integer> nums = List.of(5, 10, 15);
int total = nums.stream()
    .reduce(0, Integer::sum);
System.out.println(total);
```

Case Study Example:

```
double totalPaidRevenue = orders.stream()
    .filter(o -> o.getStatus().equalsIgnoreCase("PAID"))
    .mapToDouble(Order::getAmount)
    .sum();
```

---

4. Sorted

Simple Example:

```
List<Integer> nums = List.of(3, 7, 1, 9);
nums.stream()
```

```
    .sorted(Comparator.reverseOrder())
    .forEach(System.out::println);
```

Case Study Example:

```
List<Order> sortedOrders = orders.stream()
    .sorted(Comparator.comparingDouble(Order::getAmount).reversed())
    .toList();
```

---

5. Distinct

Simple Example:

```
List<String> names = List.of("Ravi", "Ravi", "Sita", "Amit");
names.stream()
    .distinct()
    .forEach(System.out::println);
```

Case Study Example:

```
Set<String> uniqueCustomers = orders.stream()
    .map(Order::getCustomerName)
    .collect(Collectors.toSet());
```

---

6. Limit and Skip

Simple Example:

```
List<Integer> nums = List.of(50, 20, 70, 10);
int secondHighest = nums.stream()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .get();
System.out.println(secondHighest);
```

Case Study Example:

```
Optional<Order> secondHighest = orders.stream()
    .sorted(Comparator.comparingDouble(Order::getAmount).reversed())
```

```
    .skip(1)
    .findFirst();
```

---

7. Collect

Simple Example:

```
List<Integer> nums = List.of(2, 3, 4);
List<Integer> squares = nums.stream()
    .map(n -> n * n)
    .collect(Collectors.toList());
System.out.println(squares);
```

Case Study Example:

```
List<Order> paidOrders = orders.stream()
    .filter(o -> o.getStatus().equalsIgnoreCase("PAID"))
    .collect(Collectors.toList());
```

---

8. GroupingBy

Simple Example:

```
record Student(String name, String dept) {}
List<Student> list = List.of(
    new Student("Ravi", "IT"),
    new Student("Sita", "HR"),
    new Student("Amit", "IT")
);
Map<String, List<Student>> grouped = list.stream()
    .collect(Collectors.groupingBy(Student::dept));
System.out.println(grouped);
```

Case Study Example:

```
Map<String, List<Order>> byCategory = orders.stream()
    .collect(Collectors.groupingBy(Order::getCategory));
```

---

9. PartitioningBy

Simple Example:

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);
Map<Boolean, List<Integer>> result = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
System.out.println(result);
```

Case Study Example:

```
Map<Boolean, List<Order>> partitioned = orders.stream()
    .collect(Collectors.partitioningBy(o ->
o.getStatus().equalsIgnoreCase("PAID")));
```

---

10. Optional

Simple Example:

```
List<Integer> nums = List.of();
Optional<Integer> max = nums.stream().max(Integer::compare);
System.out.println(max.orElse(0));
```

Case Study Example:

```
Optional<Order> maxOrder = orders.stream()
    .max(Comparator.comparingDouble(Order::getAmount));
System.out.println(maxOrder.orElse(null));
```

Summary of Learning Outcomes

Concept	Simple Example	Case Study Concept
filter()	Even numbers	PAID orders
map()	Uppercase names	Extract customer names
reduce()	Sum numbers	Total paid revenue
sorted()	Sort integers	Sort orders by amount

Concept	Simple Example	Case Study Concept
distinct()	Unique names	Unique customers
limit/skip	Top/second highest	Top order amount
collect()	List of squares	Collect paid orders
groupingBy()	Group by dept	Group by category
partitioningBy()	Even/Odd	Paid vs Non-Paid
Optional	Handle empty max	Highest order safe check